

Progetto di Basi di Dati

Federico Del Pup
Luigi Paşcu
Matteo Passador
Stefano Toneguzzo

26 maggio 2026

1 Indice

1. Analisi dei requisiti

- 2 Introduzione alla richiesta
- 2.2 Glossario dei termini
- 2.3 Strutturazione dei requisiti

2. 3 Progettazione concettuale

- 3.1 Gestione dell'entità Pubblicazione e delle sue specializzazioni
- 3.2 Gestione dell'entità Affiliazione e delle sue specializzazioni
- 3.3 Integrazione dei componenti e schema finale

3. 4 Progettazione logica

- 4.1 Tavola dei volumi
- 4.2 Analisi delle ridondanze
- 4.3 Ristrutturazione dello schema ER

4. 5 Progettazione fisica

- 4.4 Schema relazionale
- 5.3 Definizione di trigger per l'aggiornamento del numero di citazioni

5. 6 Query

6. ?? Conclusioni

2 Analisi dei requisiti

2.1 Introduzione alla richiesta

Il progetto richiede la progettazione di una base di dati relazionale per la gestione di una bibliografia. La base di dati deve essere in grado di gestire un ampio insieme di pubblicazioni, ciascuna caratterizzata da attributi specifici e relazioni complesse con autori, affiliazioni, editori, ecc. L'obiettivo è creare un modello concettuale che rappresenti fedelmente il dominio, seguito da una progettazione logica e fisica che ottimizzi le operazioni più frequenti, come la stampa dei dati e l'inserimento di nuove pubblicazioni. Inoltre, è richiesta l'implementazione di trigger per mantenere l'integrità dei dati e l'efficienza delle query, nonché la definizione di operazioni e query specifiche per l'analisi dei dati bibliografici.

Abbiamo valutato di focalizzare il progetto sull'ambito di una bibliografia accademica di un contesto regionale, in modo da avere un ambito limitato e ben definito.

2.2 Glossario dei termini

Termine	Descrizione	Collegamenti
Pubblicazione	Elemento di bibliografia	Autore, Pubblicazione
Articolo su rivista	Tipo di pubblicazione	Autore, Pubblicazione
Articolo per conferenza	Tipo di pubblicazione	Autore, Pubblicazione
Libro	Tipo di pubblicazione	Autore, Pubblicazione, Editore
Tesi	Tipo di pubblicazione	Autore, Pubblicazione, Università
Autore	Persona che scrive una pubblicazione	Pubblicazione, Affiliazione
Affiliazione	Gruppo di autori	Autore
Editore	Entità che pubblica un libro	Libro
Università	Luogo di discussione della tesi	Tesi

2.3 Strutturazione dei requisiti

2.3.1 Frasi di carattere generale

Si supponga di aver collezionato il seguente insieme di requisiti per la progettazione di una base di dati relazionale per la gestione di una bibliografia. Una bibliografia è costituita da un insieme di pubblicazioni.

2.3.2 Dettaglio dei requisiti

- **Pubblicazioni:** Ogni pubblicazione è caratterizzata da un codice univoco, un titolo, un anno di pubblicazione e uno o più autori. È presente una lista di riferimenti bibliografici (citazioni) verso altre pubblicazioni della bibliografia. Ogni pubblicazione tiene traccia del numero di citazioni ricevute.
- **Articoli su rivista:** Si registrano il nome della rivista, il numero del volume, le pagine (iniziale e finale) e il numero totale di pagine.

- **Articoli per conferenza:** Si registrano nome e luogo della conferenza, anno di svolgimento (indipendente dall'anno di pubblicazione), pagine (iniziale/finale) e totale pagine.
- **Libri:** Identificati dal codice ISBN, includono l'editore e il numero totale di pagine.
- **Tesi:** Si memorizzano l'argomento, l'università di discussione e il numero totale di pagine.
- **Autori:** Di ogni autore si memorizzano nome, cognome, email, pagina Web e una o più affiliazioni.
- **Affiliazioni:** Descritte da nome, indirizzo completo (via, civico, città, nazione) e numero di telefono.
- **Editori:** Descritti da nome e indirizzo fisico.
- **Università:** Identificate univocamente dal nome, includono indirizzo e telefono.

3 Progettazione concettuale

Lo schema Entità-Relazione è stato progettato per rappresentare in modo fedele e completo il dominio della bibliografia accademica, tenendo conto dei requisiti specifici e delle relazioni complesse tra le entità coinvolte. Di seguito viene presentata una descrizione dettagliata delle principali componenti dello schema ER, con particolare attenzione alla gestione delle entità e delle relazioni più rilevanti. Lo schema è stato costruito utilizzando una tecnica bottom-up, partendo dalle entità singole e dalle relazioni più semplici, per poi integrare progressivamente le specializzazioni e le associazioni più complesse. Si segnala che nello schema ER, le citazioni sono rappresentate come una relazione ricorsiva tra le pubblicazioni.

3.1 Gestione dell'entità Pubblicazione e delle sue specializzazioni

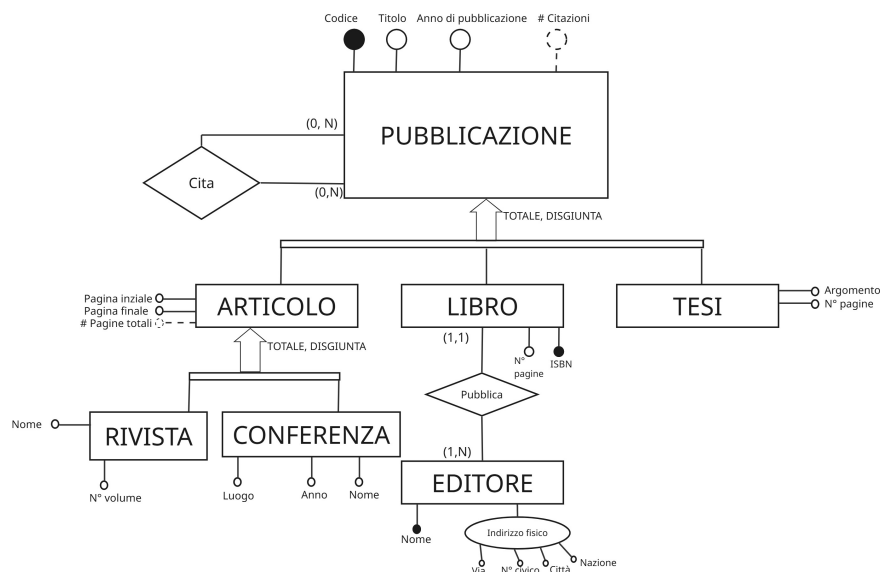


Figura 1: Entità Pubblicazione e specializzazioni

L'entità **Pubblicazione** è stata modellata come un'entità genitore con tre specializzazioni: **Articolo**, **Libro** e **Tesi**. Questa scelta è motivata dalla necessità di rappresentare in modo chiaro e strutturato le differenze tra i vari tipi di pubblicazione, che presentano attributi specifici e relazioni distinte con altre entità (ad esempio, l'editore per i libri e l'università per le tesi). La specializzazione è stata implementata utilizzando una gerarchia a copertura totale, in cui ogni pubblicazione deve essere necessariamente classificata come uno dei tre tipi specifici. L'entità **Articolo** è ulteriormente specializzata in **Rivista** e **Conferenza**, in quanto gestiscono attributi distinti (ad esempio, il numero del volume per le riviste e il luogo della conferenza per gli articoli di conferenza). La specializzazione è totale disgiunta, perchè coprono tutti gli articoli, ma non si sovrappongono tra loro. L'entità **Editore** è collegata all'entità **Libro** tramite una relazione di pubblicazione, e presenta un attributo composto *Indirizzo* che include via, numero civico, città e nazione. La relazione è uno a molti, in quanto un editore può pubblicare più libri. Infine, come scritto in precedenza, la citazione viene gestita come una relazione ricorsiva tra le pubblicazioni, consentendo di modellare in modo efficace le dinamiche di riferimento bibliografico all'interno del sistema.

3.2 Gestione dell'entità **Affiliazione** e delle sue specializzazioni

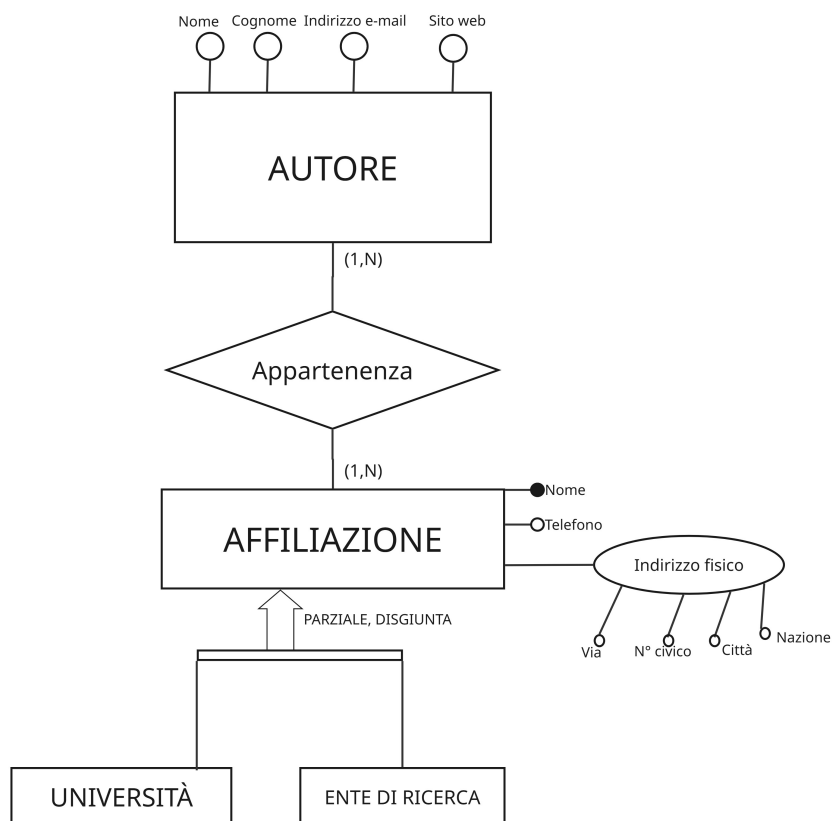


Figura 2: Entità Autore, Affiliazione e specializzazioni

L'entità **Affiliazione** è stata specializzata in **Università** ed **Ente di Ricerca** per rappresentare in modo più dettagliato le diverse tipologie di affiliazioni che un autore può avere. Questa specializzazione è stata implementata utilizzando una gerarchia a copertura parziale, in quanto ci possono essere affiliazioni che non rientrano in nessuna delle due categorie. La scelta di mantenere separate le entità figlie è motivata dalla possibilità di collegare successivamente l'università alle tesi, mentre gli enti di ricerca non hanno questa relazione. L'entità **Autore** è collegata all'entità **Affiliazione** tramite una relazione di appartenenza, che consente di modellare il fatto che un autore può essere affiliato a più enti nel tempo. La relazione è molti a molti, in quanto un autore può avere più affiliazioni e un'affiliazione può essere associata a più autori. Non è richiesto di gestire i periodi di appartenenza da parte di un autore a un'affiliazione (storicizzazione), ad esempio se un autore è stato affiliato a un'affiliazione in periodi diversi, non è necessario tenere traccia di queste informazioni.

Si noti che l'entità **Ente di Ricerca** è stata aggiunta su suggerimento del docente, al fine di rappresentare in modo più completo il dominio delle affiliazioni, che non si limitano esclusivamente alle università, ma includono anche altri tipi di organizzazioni coinvolte nella ricerca accademica.

L'entità **Autore** non presenta chiavi candidate, in quanto non è specificato dalla richiesta che qualche combinazione di attributi possa fungere da identificatore univoco.

3.3 Integrazione dei componenti e schema finale

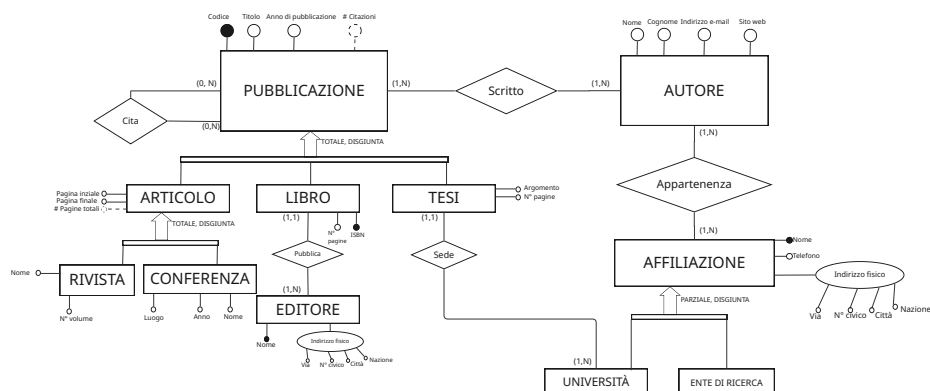


Figura 3: Schema Entità-Relazione

Per integrare i componenti prodotti, si crea una relazione molti a molti fra **Autore** e **Affiliazione**. Infine si collega l'entità **Tesi** con l'entità **Università** tramite una relazione di sede, in quanto ogni tesi è discussa in una specifica università. Questo comporta la creazione di un ciclo problematico, che viene gestito nelle fasi successive perchè potrebbero crearsi incongruenza dei dati. Es. un autore affiliato a un ente di ricerca non può essere associato a una tesi, ma se il ciclo non è gestito potrebbe essere associato a una tesi senza avere coautori affiliati all'università dove ha sede la tesi.

4 Progettazione logica

4.1 Tavola dei volumi

Concetto	Tipo	Volume
Pubblicazione	E	100.000
Articolo su rivista	E	20.000
Articolo per conferenza	E	15.000
Libro	E	10.000
Tesi	E	55.000
Autore	E	70.000
Affiliazione	E	200
Editore	E	150
Università	E	35
Ente di ricerca	E	100
Citazione	R	100.000×20
Scritto	R	$3,3 \times 100.000$
Appartenenza	R	70.000×2
Sede	R	55.000
Pubblica	R	10.000

La tavola dei volumi rappresenta il passaggio fondamentale dalla progettazione concettuale a quella logica, fornendo una stima quantitativa del carico che il database dovrà

sostenere. Il sistema gestisce un totale di **100.000 pubblicazioni**. Si noti come la somma delle entità figlie (*Articolo su rivista*, *Articolo per conferenza*, *Libro*, *Tesi*) sia esattamente pari a 100.000, riflettendo la scelta di una **gerarchia a copertura totale**. Le tesi rappresentano la quota maggioritaria (55.000), coerentemente con un dominio accademico.

Con **70.000 autori** e una media di 3,3 autori per pubblicazione (relazione **Scritto**), il sistema deve gestire circa 330.000 legami autore-opera. La relazione **Appartenenza** (70.000×2) modella correttamente il turnover accademico, prevedendo che ogni autore sia mediamente affiliato a due enti diversi nel tempo. Il numero contenuto di **Affiliazioni (200)**, **Editori (150)** e **Università (35)** suggerisce che queste tabelle fungeranno da "anagrafiche di controllo", con un basso tasso di aggiornamento rispetto alle tabelle transazionali delle pubblicazioni e citazioni. In generale questi volumi sono adatti per un contesto regionale, come quello ipotizzato. Essendo che possono esserci **Affiliazioni** che non sono né **Università** né **Enti di Ricerca**, è stato stimato un numero di 65 affiliazioni che non rientrano in nessuna delle due categorie, portando il totale a 200.

4.2 Analisi delle ridondanze

Frequenza delle Operazioni più comuni

Ecco una stima della frequenza delle operazioni più comuni che il sistema dovrà gestire, basata su ipotesi ragionevoli e coerenti con il dominio accademico regionale:

OP1 Stampa delle pubblicazioni

Frequenza: 200/500 volte al giorno.

OP2 Inserimento e Aggiornamento

Inserire una pubblicazione (40 volte al giorno).

Aggiornamento delle citazioni di una pubblicazione (40 volte al giorno)

OP3 Inserire un autore

Frequenza: 10 volte al giorno.

OP4 Modifica affiliazione

Frequenza: 1 volta al giorno (frequenza stimata).

Gli unici due attributi ridondanti presenti nello schema sono: **NumeroCitazioni** nell'entità **Pubblicazione** e **NumeroPagineTotali** nell'entità **Articolo**. Si fa un'analisi comparativa tra le due soluzioni per il primo attributo, considerando l'impatto della ridondanza sull'efficienza del sistema, in particolare per le operazioni di stampa e aggiornamento delle citazioni, che sono le più frequenti. Per il secondo attributo ridondante, essendo un attributo comune a tutte le sottoentità dell'entità **Pubblicazione**, si è deciso di mantenerlo. Da qui si è optato di rimuovere l'attributo **Pagina finale**, in quanto è facilmente calcolabile a partire da **Pagina iniziale** e **NumeroPagineTotali**.

4.2.1 Con Ridondanza dell'attributo NumeroCitazioni

Per il calcolo del costo, si assume che un accesso a un'entità o relazione di tipo "L" (lettura) abbia un costo unitario di 1, mentre un accesso di tipo "S" (scrittura) abbia un costo unitario di 2, riflettendo l'overhead aggiuntivo associato alle operazioni di scrittura rispetto a quelle di lettura. La stima del costo totale è effettuata con il dato di frequenza

maggiore per ogni operazione, al fine di valutare l'impatto massimo della ridondanza sull'efficienza del sistema.

Operazione 1

Concetto	Costrutto	Accessi	Tipo
Pubblicazione	Entità	1	L

Costo: $(500 \times 1) \times 1 = 500$ unità di costo

Operazione 2

Concetto	Costrutto	Accessi	Tipo
Pubblicazione	Entità	2	S
Cita	Relazione	1	S
Pubblicazione	Entità	1	L
Autore	Entità	1	S

Passi logici:

- 1) Salvo i dati della pubblicazione
- 2) Salvo le coppie di pubblicazione e pubblicazione citata
- 3) Cerco la pubblicazione citata
- 4) Incremento di 1 il numero di citazioni a quella pubblicazione
- 5) Salvo la coppia pubblicazione autore

Costo: $(40 \times 4) \times 2 + (40 \times 1) \times 1 = 360$ unità

4.2.2 Senza Ridondanza dell'attributo NumeroCitazioni

Operazione 1

Concetto	Costrutto	Accessi	Tipo
Pubblicazione	Entità	1	L
Citazione	Relazione	20	L

Descrizione: Stampo i dati relativi ad una pubblicazione. Per ogni pubblicazione, per calcolare il numero di citazioni si accede alla relazione Citazione. Il numero medio di accessi per Pubblicazione è calcolato come: $\frac{\text{volume Citazione}}{\text{volume Pubblicazioni}} = 20$.

Costo: $500 \times (20 + 1) \times 1 = 10.500$ unità di costo

Operazione 2

Concetto	Costrutto	Accessi	Tipo
Pubblicazione	Entità	1	S
Cita	Relazione	1	S
Autore	Entità	1	S

Passi logici:

- 1) Salvo i dati della pubblicazione
- 2) Salvo le coppie di pubblicazione e pubblicazione citata
- 3) Salvo la coppia pubblicazione autore

Costo: $40 \times 3 \times 2 = 240$ unità di costo

4.2.3 Studio Comparativo

Costo	Presenza di ridondanza	Assenza di ridondanza
OP1	500	10.500
OP2	360	240
Totale	860	10.740

Conclusione: Poiché $860 < 10.740$, conviene mantenere l'attributo derivato.

4.3 Ristrutturazione dello schema ER

In seguito vengono mostrate le modifiche apportate al modello ER per favorire la traduzione nello schema logico relazionale, con particolare attenzione alla gestione delle specializzazioni e alla semplificazione di alcune strutture complesse.

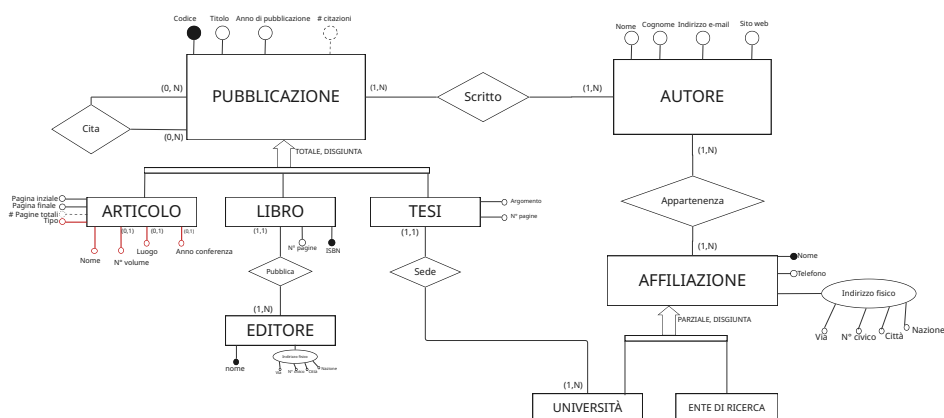


Figura 4: Schema ER modificato dopo la prima revisione

Specializzazione Rivista-Conferenza: La gerarchia è stata accorpata verso l'alto (ristrutturazione verso il padre) eliminando le entità figlie e migrandone gli attributi nell'entità genitore *Articolo*. Per distinguere le occorrenze, è stato introdotto l'attributo discriminante *Tipo*, i cui valori sono vincolati al dominio {Rivista, Conferenza}.

Gli attributi derivanti dalle classi figlie sono stati definiti come opzionali (cardinalità 0..1), con l'eccezione dell'attributo *Nome*, che rimane obbligatorio in quanto comune a entrambe le tipologie. Si specifica, inoltre, che l'attributo *AnnoConferenza* deve essere semanticamente distinto dall'attributo *AnnoPubblicazione*. Ad esempio una Conferenza può essere svolta in un anno diverso da quello di pubblicazione dell'articolo associato.

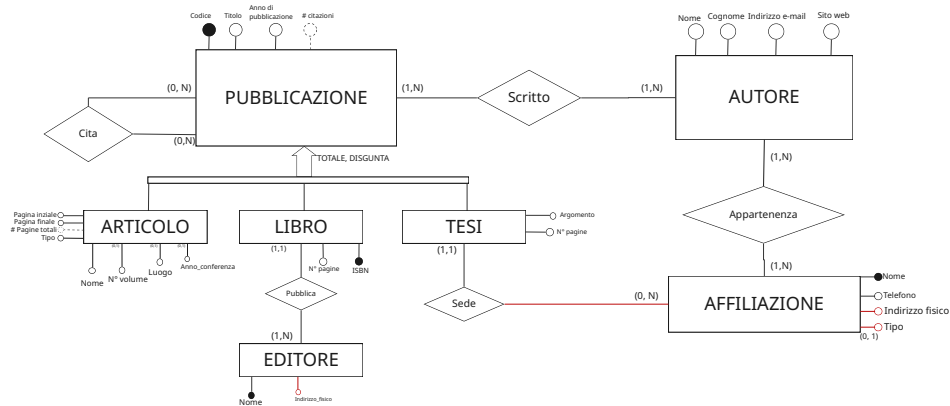


Figura 5: Schema ER modificato dopo la seconda revisione

Semplificazione della Gerarchia Affiliazione: La specializzazione tra l'entità *Affiliazione* (padre) e le sottoentità *Università* ed *Ente di Ricerca* è stata risolta mediante l'accorpamento verso il padre. Tale scelta è motivata dall'assenza di attributi specifici rilevanti nelle entità figlie.

Per preservare la semantica originale, è stato introdotto un vincolo di integrità basato sul tipo di affiliazione: l'associazione con l'entità *Tesi* è valida esclusivamente per le occorrenze di tipo *Università*. In termini di cardinalità, il rapporto tra *Tesi* ed *Affiliazione* è di (1,1), mentre la partecipazione dell'entità *Affiliazione* è (0,n), poiché solo il sottoinsieme relativo alle università può essere associato a una tesi.

Semplificazione dell'attributo Indirizzo: L'attributo composto *Indirizzo Fisico* è stato ristrutturato mediante un processo di aggregazione dei suoi componenti (quali via, numero civico, CAP e città). Al fine di semplificare l'implementazione nel modello logico e facilitare le operazioni di inserimento, i singoli campi sono stati unificati in un unico attributo atomico di tipo stringa. Questo viene fatto perché non esistono gli attributi composti nel modello logico relazionale, quindi è necessario semplificare la struttura dell'attributo per garantire una traduzione più diretta e senza perdita di informazioni.

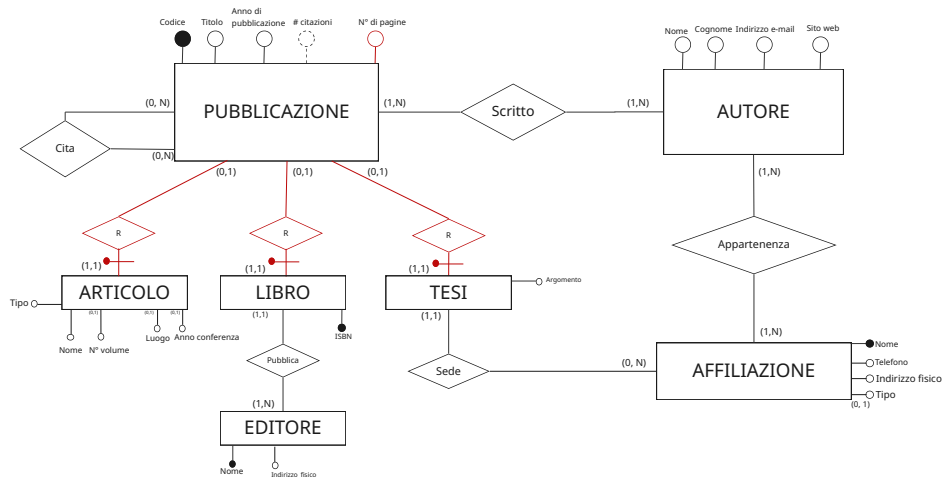


Figura 6: Schema ER modificato dopo la terza revisione

Risoluzione della Gerarchia Pubblicazione: La specializzazione tra l'entità genitore *Pubblicazione* e le sottoclassi *Articolo*, *Libro* e *Tesi* è stata risolta mantenendo le entità distinte (strategia delle entità multiple).

Per implementare correttamente il legame logico, sono state definite relazioni esplicite tra il padre e i figli tramite l'uso di chiavi esterne (*foreign keys*) nelle entità figlie che referenziano la chiave primaria di *Pubblicazione*. Al fine di garantire la copertura totale della specializzazione, è stato introdotto un vincolo di integrità aggiuntivo: ogni occorrenza dell'entità *Pubblicazione* deve necessariamente essere referenziata come chiave esterna in almeno una delle tre entità figlie.

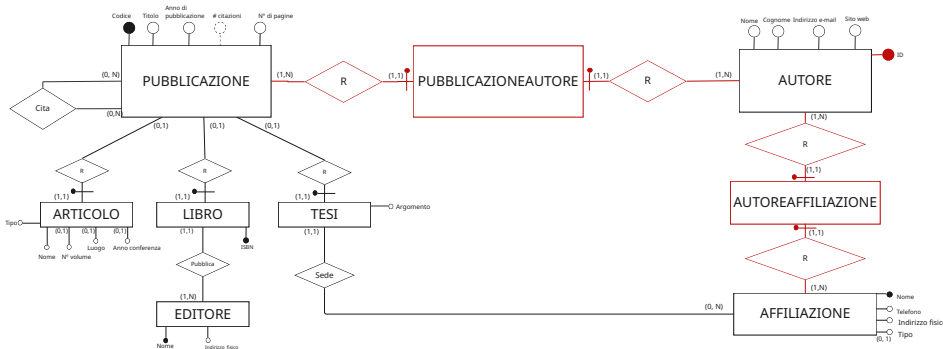


Figura 7: Schema ER modificato dopo la quarta revisione

Ristrutturazione della relazione "Scritto" e "Appartenenza", chiave su Autore: La relazione *Scritto* è stata ristrutturata in modo da inserire un'entità associativa, denominata *Pubb-Autori*, che collega l'entità *Autore* con l'entità *Pubblicazione*. Questa scelta è motivata dalla necessità di rappresentare correttamente la relazione multi-a-multi tra autori e pubblicazioni, consentendo di gestire efficacemente i casi in cui una pubblicazione abbia più autori e un autore abbia più pubblicazioni. Lo stesso approccio è stato adottato per la relazione *Appartenenza*, con l'introduzione dell'entità associativa *Aut-Aff* che collega l'entità *Autore* con l'entità *Affiliazione*. Questa ristrutturazione consente di modellare in modo più flessibile le dinamiche di affiliazione degli autori, che possono essere associati a più enti nel tempo, e di mantenere una chiara tracciabilità delle relazioni tra

autori e affiliazioni.

Infine, si è notato che l'entità *Autore* non presenta chiavi candidate, in quanto non è specificato dalla richiesta che qualche combinazione di attributi possa fungere da identificatore univoco. Pertanto, è stato introdotto un nuovo attributo *ID* come chiave primaria per l'entità *Autore*, garantendo così l'univocità.

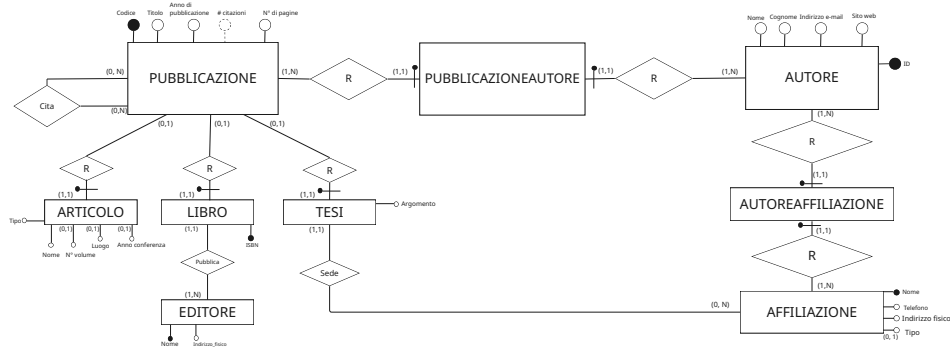


Figura 8: Schema ER finale senza le modifiche di dettaglio

Si evidenzia che dal modello ER non si riescono a gestire i vincoli di non autocitazione e non citazione circolare. Anche il ciclo problematico creato dalla relazione tra *Pubblicazione*, *Autore* e *Affiliazione* non è gestibile. Questi vincoli saranno gestiti a livello di progettazione fisica, tramite l'implementazione di trigger che assicurino la coerenza dei dati durante le operazioni di inserimento e aggiornamento.

4.4 Schema logico

Dal modello ER finale, si ottiene lo schema logico relazionale seguente:

PUBBLICAZIONI(Codice, Titolo, Anno, NumeroPagine, NumeroCitazioni)

ARTICOLI(CodicePubblicazione, Nome, NumeroVolume, LuogoConferenza, AnnoConferenza, PaginaIniziale, Tipo)

LIBRI(CodicePubblicazione, ISBN, NomeEditore)

TESI(CodicePubblicazione, Argomento, Affiliazione)

AUTORI(ID, Nome, Cognome, Email, SitoWeb)

AFFILIAZIONI(Nome, Indirizzo, Telefono, Tipo)

EDITORI(Nome, Indirizzo)

CITAZIONI(PubblicazioneCitante, PubblicazioneCitata)

PUBBLICAZIONEAUTORE(IDAutore, CodicePubblicazione)

AUTOREAFFILIAZIONE(IDAutore, NomeAffiliazione)

*le chiavi primarie sono sottolineate, mentre le chiavi esterne sono indicate con frecce.

Chiavi esterne:

ARTICOLI.CodicePubblicazione $\rightarrow$ PUBBLICAZIONI.Codice

LIBRI.CodicePubblicazione $\rightarrow$ PUBBLICAZIONI.Codice

TESI.CodicePubblicazione $\rightarrow$ PUBBLICAZIONI.Codice

CITAZIONI.PubblicazioneCitante $\rightarrow$ PUBBLICAZIONI.Codice

CITAZIONI.PubblicazioneCitata $\rightarrow$ PUBBLICAZIONI.Codice

LIBRI.NomeEditore $\rightarrow$ EDITORI.Nome

TESI.Affiliazione $\rightarrow$ AFFILIAZIONI.Nome

PUBBLICAZIONEAUTORE.IDAutore $\rightarrow$ AUTORI.ID

PUBBLICAZIONEAUTORE.CodicePubblicazione → PUBBLICAZIONI.Codice
AUTOREAFFILIAZIONE.IDAutore → AUTORI.ID
AUTOREAFFILIAZIONE.NomeAffiliazione → AFFILIAZIONI.Nome

Si noti che le chiavi primarie sono state definite in modo da garantire l'univocità di ogni record, mentre le chiavi esterne sono state implementate per mantenere l'integrità referenziale tra le tabelle, assicurando che le relazioni logiche tra i dati siano rispettate. L'unica chiave candidabile non scelta come chiave primaria è l'ISBN nella tabella **LIBRI**, poiché il codice della pubblicazione è già un identificatore univoco sufficiente per distinguere ogni libro all'interno del sistema. Si è preferito mantenere un'unica chiave primaria per semplificare le operazioni di join e ridurre la complessità del modello, evitando la necessità di gestire chiavi composite o multiple per la stessa entità.

5 Progettazione fisica (Implementazione DB)

Il DB è stato implementato utilizzando il sistema di gestione di database relazionale PostgreSQL, scelto per la sua robustezza, scalabilità e supporto avanzato alle funzionalità richieste dal progetto. La progettazione fisica ha seguito le best practice per l'ottimizzazione delle prestazioni, la gestione dell'integrità dei dati e la facilità di manutenzione. Di seguito vengono descritte le principali fasi dell'implementazione, inclusa la generazione delle tabelle SQL, il popolamento del database con dati di esempio e l'implementazione di trigger per garantire la coerenza dei dati.

5.1 Generazione delle tabelle SQL

Di seguito viene presentata la sintassi SQL utilizzata per creare le tabelle del database, in Data Definition Language (DDL). La definizione delle tabelle include la specifica delle chiavi primarie, chiavi esterne e vincoli di integrità per garantire la coerenza dei dati e il rispetto delle relazioni logiche tra le entità.

```
CREATE TABLE public.affiliazioni
(
    nome varchar(100) NOT NULL,
    telefono varchar(100) NOT NULL,
    indirizzo varchar(100) NOT NULL,
    tipo varchar(20),
    CONSTRAINT affiliazione_pkey PRIMARY KEY (nome)
)

CREATE TABLE IF NOT EXISTS public.articoli
(
    CodicePubblicazione integer NOT NULL,
    tipo varchar(50) NOT NULL,
    nome varchar(255) NOT NULL,
    Npaginainiziale integer,
    Nvolume integer,
    luogoConferenza varchar(100),
    annoConferenza integer,
    CONSTRAINT articolo_pkey PRIMARY KEY (CodicePubblicazione ),
```

```

        CONSTRAINT articolo_codice_pubblicazione_fkey FOREIGN KEY (
            ↳ CodicePubblicazione )
REFERENCES public.pubblicazioni (codice) MATCH SIMPLE
        ON UPDATE NO ACTION
        ON DELETE CASCADE
    )

CREATE TABLE public.autoreaaffiliazione
(
    idAutore integer NOT NULL,
    nomeAffiliazione varchar(255) NOT NULL,
    CONSTRAINT auto_aff_pkey PRIMARY KEY (id_autore,
        ↳ nome_affiliazione),
    CONSTRAINT auto_aff_id_autore_fkey FOREIGN KEY (id_autore)
        REFERENCES public.autori (id) MATCH SIMPLE
        ON UPDATE CASCADE
        ON DELETE CASCADE,
    CONSTRAINT auto_aff_nome_affiliazione_fkey FOREIGN KEY (
        ↳ nome_affiliazione)
        REFERENCES public.affiliazioni (nome) MATCH SIMPLE
        ON UPDATE CASCADE
        ON DELETE CASCADE
)

CREATE TABLE public.autori
(
    id SERIAL NOT NULL,
    nome varchar(100) NOT NULL,
    cognome varchar(100) NOT NULL,
    email varchar(100) NOT NULL,
    sitoweb varchar(100) NOT NULL,
    CONSTRAINT autore_pkey PRIMARY KEY (id)
)

CREATE TABLE public.citazioni
(
    pubblicazioneCitante integer NOT NULL,
    pubblicazioneCitata integer NOT NULL,
    CONSTRAINT citazione_pkey PRIMARY KEY (pubblicazioneCitante,
        ↳ pubblicazioneCitata),
    CONSTRAINT citazione_codice_pubblicazione_citante_fkey FOREIGN
        ↳ KEY (pubblicazioneCitante )
        REFERENCES public.pubblicazioni (codice) MATCH SIMPLE
        ON UPDATE NO ACTION
        ON DELETE CASCADE,
    CONSTRAINT citazione_codice_pubblicazione_citata_fkey FOREIGN
        ↳ KEY (pubblicazioneCitata )
        REFERENCES public.pubblicazioni (codice) MATCH SIMPLE
        ON UPDATE NO ACTION
        ON DELETE CASCADE
)

```

```

)

CREATE TABLE public.editori
(
    nome varchar(100) NOT NULL,
    indirizzo varchar(255) NOT NULL,
    CONSTRAINT editore_pkey PRIMARY KEY (nome)
)

CREATE TABLE public.libri
(
    CodicePubblicazione integer NOT NULL,
    isbn varchar(17) NOT NULL,
    nomeEditore varchar(255) NOT NULL,
    CONSTRAINT libro_pkey PRIMARY KEY (CodicePubblicazione ),
    CONSTRAINT libro_codice_pubblicazione_fkey FOREIGN KEY (
        ↪ CodicePubblicazione )
        REFERENCES public.pubblicazioni (codice) MATCH SIMPLE
        ON UPDATE NO ACTION
        ON DELETE CASCADE,
    CONSTRAINT libro_nome_editore_fkey FOREIGN KEY (nomeEditore)
        REFERENCES public.editori (nome) MATCH SIMPLE
        ON UPDATE NO ACTION
        ON DELETE NO ACTION
);

CREATE TABLE public.pubblicazioneautore
(
    CodicePubblicazione integer NOT NULL,
    idAutore integer NOT NULL,
    CONSTRAINT pubb_autore_pkey PRIMARY KEY (CodicePubblicazione,
        ↪ idAutore),
    CONSTRAINT pubb_autore_codice_pubblicazione_fkey FOREIGN KEY (
        ↪ CodicePubblicazione )
        REFERENCES public.pubblicazioni (codice) MATCH SIMPLE
        ON UPDATE NO ACTION
        ON DELETE CASCADE,
    CONSTRAINT pubb_autore_id_autore_fkey FOREIGN KEY (idAutore)
        REFERENCES public.autori (id) MATCH SIMPLE
        ON UPDATE NO ACTION
        ON DELETE CASCADE
);

CREATE TABLE public.pubblicazioni
(
    codice SERIAL NOT NULL,
    titolo varchar(200),
    annopubblicazione integer NOT NULL,
    ncitazioni integer DEFAULT 0,
    npagine integer NOT NULL,
    CONSTRAINT pubblicazione_pkey PRIMARY KEY (codice)
)

```

```

)

CREATE TABLE public.tesi
(
    CodicePubblicazione integer NOT NULL,
    argomento text NOT NULL,
    nomeAffiliazione varchar(255) NOT NULL,
    CONSTRAINT tesi_pkey PRIMARY KEY (CodicePubblicazione ),
    CONSTRAINT tesi_codice_pubblicazione_fkey FOREIGN KEY (
        ↪ CodicePubblicazione )
        REFERENCES public.pubblicazioni (codice) MATCH SIMPLE
        ON UPDATE NO ACTION
        ON DELETE CASCADE,
    CONSTRAINT tesi_nome_affiliazione_fkey FOREIGN KEY (
        ↪ nomeAffiliazione)
        REFERENCES public.affiliazioni (nome) MATCH SIMPLE
        ON UPDATE NO ACTION
        ON DELETE NO ACTION
)

```

5.2 Popolamento del database

Per popolare il database con dati di esempio, sono stati creati script SQL che inseriscono record nelle tabelle in modo coerente con le relazioni e i vincoli definiti nello schema logico. I dati di esempio sono stati generati in modo da riflettere scenari realistici nel contesto accademico, includendo una varietà di pubblicazioni, autori, affiliazioni e citazioni. Abbiamo scelto come contesto regionale l'area del Triveneto, che comprende le regioni del Veneto, Friuli-Venezia Giulia e Trentino-Alto Adige. Questo contesto è stato selezionato per la sua ricca attività accademica e di ricerca, che offre un'ampia gamma di dati per testare le funzionalità del database. Inoltre è coerente con i volumi stimati nella tavola dei volumi, in quanto si tratta di un contesto regionale che presenta una significativa attività accademica e di pubblicazione scientifica.

Ecco un esempio di tabella risultante dal popolamento del database:

Tabella 1: Affiliazione			
Affiliazione			
Nome	Telefono	Indirizzo	Tipo
Università degli Studi di Udine	0432-556111	Via Palladio 8, Udine	Università
Università degli Studi di Trieste	040-5587111	Piazzale Europa 1, Trieste	Università
SISSA	040-3787111	Via Bonomea 265, Trieste	Università
Università degli Studi di Padova	049-8275111	Via 8 Febbraio 2, Padova	Università
Università Ca' Foscari Venezia	041-2345111	Dorsoduro 3246, Venezia	Università
Area Science Park	040-3755111	Padriciano 99, Trieste	Ente di ricerca
OGS	040-21401	Borgo Grotta Gigante 42/C, Sgonico	Ente di ricerca
CRO Aviano	0434-659111	Via Franco Gallini 2, Aviano	Ente di ricerca

5.3 Trigger

Proprietà logiche dei vincoli sulle citazioni

Non Autocitazione: Una pubblicazione non può citare sé stessa. Questo è il vincolo più elementare e serve a prevenire l'inserimento di dati logicamente inconsistenti, garantendo che il conteggio delle citazioni rifletta solo i riferimenti esterni.

Non Citazione Circolare: Una pubblicazione A non può citare una pubblicazione B se B cita già A, direttamente o indirettamente. Questo vincolo impedisce la formazione di cicli di citazione che potrebbero distorcere il conteggio delle citazioni e complicare l'analisi bibliometrica.

Nota sul Vincolo Temporale: Una pubblicazione non può citare un'altra pubblicazione con un anno di pubblicazione successivo. Pur riconoscendo l'esistenza di casi come i preprint nel mondo reale questo vincolo è assunto logicamente come rilevante ed è implementato nel database.

Trigger per l'incremento del numero di citazioni

Il trigger `inc_n_citazioni` è progettato per gestire l'incremento del numero di citazioni di una pubblicazione ogni volta che viene inserita una nuova citazione che la riguarda. Il trigger esegue le seguenti operazioni: 1. Verifica che la pubblicazione citante non sia la stessa della pubblicazione citata, prevenendo così l'autocitazione. 2. Controlla che la pubblicazione citante non stia citando una pubblicazione con un anno di pubblicazione successivo, garantendo la coerenza temporale delle citazioni. 3. Se i controlli sono superati, il trigger incrementa il numero di citazioni della pubblicazione citata, aggiornando così l'attributo derivato **NumeroCitazioni** in modo accurato e tempestivo.

```
--1)modifica num di citazioni
--Aggiunta del num di citazioni
--PRIMA O DOPO L'INSERIMENTO???
CREATE OR REPLACE FUNCTION inc_n_citazioni()
  RETURNS TRIGGER LANGUAGE plpgsql AS $$
  DECLARE
    n integer;
    Anno_citante integer;
    Anno_citata integer;
  BEGIN
    SELECT Anno INTO Anno_citante
    FROM PUBBLICAZIONI
    WHERE Codice = NEW.PubblicazioneCitante;

    SELECT Anno, NumeroCitazioni INTO Anno_citata, n
    FROM PUBBLICAZIONI
    WHERE Codice = NEW.PubblicazioneCitata;

    IF NEW.PubblicazioneCitante = NEW.PubblicazioneCitata THEN
      RAISE EXCEPTION 'Una pubblicazione non si può citare da
        ↪ sola';
    END IF;
    IF Anno_citante < Anno_citata THEN
```

```

        RAISE EXCEPTION 'Una pubblicazione può citare solo
        ↳ pubblicazioni contemporanee o più vecchie';
ELSE
    IF n = 0 THEN
        UPDATE PUBBLICAZIONI
        SET NumeroCitazioni = 1
        WHERE Codice = NEW.PubblicazioneCitata;
    ELSE
        UPDATE PUBBLICAZIONI
        SET NumeroCitazioni = n+1
        WHERE Codice = NEW.PubblicazioneCitata;
    END IF;
    RETURN NEW;
END IF;
END;
$$;
--SELECT count(*) INTO n
--
--                                FROM CITAZIONE
--                                WHERE EXISTS(SELECT * FROM PUBBLICAZIONE
        ↳ WHERE NEW.codice_pubblicazione_citata = codice) AND NEW.
        ↳ codice_pubblicazione_citata = codice_pubblicazione_citata;

```

5.4 Trigger per la rimozione del numero di citazioni

```

--Rimozione del num di citazioni
CREATE OR REPLACE FUNCTION dec_n_citazioni()
    RETURNS TRIGGER
    LANGUAGE plpgsql AS $$
    DECLARE
        n integer;
    BEGIN
        SELECT NumeroCitazioni INTO n
        FROM PUBBLICAZIONI
        WHERE Codice = OLD.PubblicazioneCitata;

        IF NOT FOUND THEN
            RETURN NULL;
        ELSE
            UPDATE PUBBLICAZIONI
            SET NumeroCitazioni = n-1
            WHERE Codice = OLD.PubblicazioneCitata;
        END IF;
        RETURN OLD;
    END;
$$;

```

Trigger per l'update delle citazioni

Il trigger controlla che una pubblicazione non si citi da sola e gestisce l'attributo derivato **NumeroCitazioni**

```

--Update citazioni, controlla che non si citi da sola e gestisce il
    ↳ campo derivato
--HA SENSO???
CREATE or REPLACE FUNCTION updt_citazioni()
    RETURNS trigger LANGUAGE plpgsql AS $$
BEGIN
    RAISE EXCEPTION 'Una citazione non puo' essere
        ↳ aggiornata si consiglia di aggiungerne una
        ↳ nuova ed eliminare la vecchia';
END;
$$;

CREATE or REPLACE TRIGGER trg_ins_CIT
BEFORE INSERT ON CITAZIONI
FOR EACH ROW EXECUTE FUNCTION inc_n_citazioni();

CREATE or REPLACE TRIGGER trg_dec_CIT
BEFORE DELETE ON CITAZIONI
FOR EACH ROW EXECUTE FUNCTION dec_n_citazioni();

CREATE or REPLACE TRIGGER trg_upd_CIT
BEFORE UPDATE ON CITAZIONI
FOR EACH ROW EXECUTE FUNCTION updt_citazioni();

```

Trigger per il controllo dell'affiliazione universitaria nelle tesi

Il trigger `trg_TESI` è progettato per garantire che ogni tesi inserita nel database sia associata a un'affiliazione di tipo "Università" e che questa affiliazione sia effettivamente collegata ad almeno uno degli autori della tesi. Questo controllo è fondamentale per mantenere l'integrità dei dati e assicurare che le tesi siano correttamente categorizzate all'interno del contesto accademico.

Il trigger esegue le seguenti verifiche: 1. Controlla che il codice della pubblicazione associato alla tesi non sia già utilizzato in altre pubblicazioni (articoli o libri), prevenendo così duplicazioni. 2. Verifica che l'affiliazione selezionata per la tesi sia effettivamente un'università presente nel database. 3. Assicura che l'affiliazione universitaria sia collegata ad almeno uno degli autori della tesi, garantendo così la coerenza tra l'affiliazione e i contributori della pubblicazione.

```

--3)Check dell'uni come affiliazione
--Prima inserimento effettivo TESI, controllo che l'affiliazione
    ↳ selezionata sia un'università
CREATE OR REPLACE FUNCTION check_uni()
    RETURNS TRIGGER
    LANGUAGE plpgsql AS
$$
BEGIN
    IF EXISTS(SELECT CodicePubblicazione FROM ARTICOLI WHERE NEW.
        ↳ CodicePubblicazione = CodicePubblicazione)

```

```

OR EXISTS(SELECT CodicePubblicazione FROM LIBRI WHERE NEW.
    ↳ CodicePubblicazione = CodicePubblicazione) THEN
    RAISE EXCEPTION 'Il codice della pubblicazione è
        ↳ attualmente in uso';
    RETURN NULL;
END IF;

IF NOT EXISTS(SELECT Nome FROM AFFILIAZIONI WHERE NEW.
    ↳ NomeAffiliazione = Nome AND Tipo = 'Università') THEN
    RAISE EXCEPTION 'Università inserita non corretta o non
        ↳ presente nel db';
END IF;

IF NEW.NomeAffiliazione NOT IN(SELECT AA.NomeAffiliazione
                                FROM PUBBLICAZIONEAUTORE PA,
                                ↳ AUTOREAFFILIAZIONE AA
                                WHERE PA.CodicePubblicazione = NEW.
                                    ↳ CodicePubblicazione AND AA.
                                    ↳ IDAutore = PA.IDAutore) THEN
    RAISE EXCEPTION 'Università inserita non è affiliata
        ↳ neanche ad un autore';
END IF;
    RETURN NEW;
END;
$$;

CREATE or REPLACE TRIGGER trg_TESI
BEFORE INSERT OR UPDATE ON TESI
FOR EACH ROW EXECUTE FUNCTION check_uni();

```

6 Query

Di seguito vengono presentate le query SQL per rispondere alle domande specifiche richieste

Q1 Autore con più pubblicazioni per ogni affiliazione

Stampare il nome dell'autore che detiene il maggior numero di pubblicazioni per ogni singola affiliazione.

Q2 Co-autoria esclusiva

Stampare le coppie di autori che hanno collaborato sempre e solo tra di loro (ovvero, ogni loro pubblicazione è stata fatta in coppia).

Q3 Autori fedeli all'affiliazione

Stampare gli autori che non hanno mai prodotto pubblicazioni con co-autori appartenenti a un'affiliazione diversa dalla propria.

Q4 Analisi dell'impatto a breve termine

Trovare gli autori che hanno pubblicato esclusivamente articoli i quali, a distanza di 2 anni dalla data di pubblicazione, hanno accumulato almeno 5 citazioni.

Q1: Autore con più pubblicazioni per ogni affiliazione

Stampare il nome dell'autore che detiene il maggior numero di pubblicazioni per ogni singola affiliazione.

```
SELECT a.nome, a.cognome, aa.nome_affiliazione
FROM AUTORE a
JOIN PUBB_AUTORE pa ON a.id = pa.id_autore
JOIN AUTO_AFF aa ON a.id = aa.id_autore
GROUP BY a.id, a.nome, a.cognome, aa.nome_affiliazione
HAVING COUNT(pa.codice_pubblicazione) = (
    SELECT MAX(cnt)
    FROM (
        SELECT COUNT(pa2.codice_pubblicazione) AS cnt
        FROM AUTORE a2
        JOIN PUBB_AUTORE pa2 ON a2.id = pa2.id_autore
        JOIN AUTO_AFF aa2 ON a2.id = aa2.id_autore
        WHERE aa2.nome_affiliazione = aa.nome_affiliazione
        GROUP BY a2.id
    ) sub
);
```

Q2: Coppie che hanno pubblicato sempre e solo insieme

Stampare le coppie di autori che hanno collaborato sempre e solo tra di loro (ovvero, ogni loro pubblicazione è stata fatta in coppia).

```
SELECT pa1.id_autore, pa2.id_autore
FROM PUBB_AUTORE pa1
JOIN PUBB_AUTORE pa2
    ON pa1.codice_pubblicazione = pa2.codice_pubblicazione
    AND pa1.id_autore < pa2.id_autore
GROUP BY pa1.id_autore, pa2.id_autore
HAVING COUNT(DISTINCT pa1.codice_pubblicazione) = (
    SELECT COUNT(*)
    FROM PUBB_AUTORE pa
    WHERE pa.id_autore = pa1.id_autore
)
AND COUNT(DISTINCT pa2.codice_pubblicazione) = (
    SELECT COUNT(*)
    FROM PUBB_AUTORE pa
    WHERE pa.id_autore = pa2.id_autore
);
```

Q3: Autori che non hanno mai pubblicato con colleghi di altre affiliazioni

Stampare gli autori che non hanno mai prodotto pubblicazioni con co-autori appartenenti a un'affiliazione diversa dalla propria.

```
SELECT a.id, a.nome, a.cognome
FROM AUTORE a
WHERE NOT EXISTS (
    SELECT 1
    FROM PUBB_AUTORE pa1
    JOIN PUBB_AUTORE pa2
        ON pa1.codice_pubblicazione = pa2.codice_pubblicazione
        AND pa1.id_autore <> pa2.id_autore
    JOIN AUTO_AFF aa1
        ON aa1.id_autore = pa1.id_autore
    JOIN AUTO_AFF aa2
        ON aa2.id_autore = pa2.id_autore
    WHERE pa1.id_autore = a.id
        AND aa1.nome_affiliazione <> aa2.nome_affiliazione
);
```

Questa

Q4: Autori con solo articoli di successo (min. 5 citazioni a 2 anni)

Trovare gli autori che hanno pubblicato esclusivamente articoli i quali, a distanza di 2 anni dalla data di pubblicazione, hanno accumulato almeno 5 citazioni.

```
SELECT a.id, a.nome, a.cognome
FROM public.autore a
-- Condizione A: L'autore deve aver pubblicato almeno un articolo
WHERE EXISTS (
    SELECT 1 FROM public.pubb_autore pa WHERE pa.id_autore = a.id
)
-- Condizione B: NON deve esistere alcuna sua pubblicazione "flop"
AND NOT EXISTS (
    SELECT 1
    FROM public.pubb_autore pa
    JOIN public.pubblicazione p ON pa.codice_pubblicazione = p.
        ↳ codice
    WHERE pa.id_autore = a.id
        AND (
            SELECT COUNT(*)
            FROM public.citazione c
            JOIN public.pubblicazione pc ON c.
                ↳ codice_pubblicazione_citante = pc.codice
            WHERE c.codice_pubblicazione_citata = p.codice
                AND (pc.annopubblicazione - p.annopubblicazione) <= 2
        ) < 5
);
```

Nota sull'inefficacia dell'attributo derivato nella query OP8: Nonostante la presenza dell'attributo derivato **NumeroCitazioni** nell'entità **PUBBLICAZIONE** — introdotto per ottimizzare la stampa dei dati generali (*OP1*) — esso risulta inutilizzabile ai fini della query **OP8** (*Analisi dell'impatto a breve termine*).

Il limite risiede nella natura stessa dell'attributo ridondante:

- **Staticità del dato:** L'attributo memorizza esclusivamente il conteggio totale e cumulativo delle citazioni ricevute fino all'istante attuale.
- **Vincolo Temporale Dinamico:** La specifica della *OP8* richiede il filtraggio delle sole citazioni avvenute entro un arco temporale di **due anni** dalla data di rilascio della pubblicazione.

Di conseguenza, per soddisfare il requisito, è necessario ignorare il valore pre-calcolato ed effettuare un calcolo dinamico accedendo direttamente alla relazione **CITAZIONI**. Questo processo permette di confrontare gli attributi **Anno** della pubblicazione citante e di quella citata, operazione indispensabile per isolare il sottoinsieme di riferimenti bibliografici pertinenti all'analisi richiesta.